

Document number: P0382R0
Date: 2016-05-29
Project: Programming Language C++, Evolution Working Group
Reply-to: Tomasz Kamiński <tomaszkam at gmail dot com>

Comments on P0119: Overload sets as function arguments

1. Introduction

The aim of this paper is to present the authors comments on the impact of the change proposed in [P0119R1: Overload sets as function arguments](#). At the begin I would like to clarify that I see a great need for a language level feature that would allow programmers to easily create an closure that wraps polymorphic calls to function with given name.

Observations and comments gathered in this paper are related to only one of aspect of the feature, that makes the compiler responsible for decision in which situation the closure generation will actually be performed. As consequence all of them would be voided by design, that would allow programmer explicitly request the generation of the functor object. This may be achieved, for example by using `[] id-expression` syntax proposed in [N3617: Lifting overload sets into function objects](#) while keeping the same rules for functor generation.

2. Fragility of the program semantics

[P0119R1](#) proposes that *id-expression* passed as argument for an unconstrained template parameter means two different things based on whether *id-expression* refers to a unique function or a function overload set, as seen at the point of its processing. In other words, whether we intepret *id-expression* as a pointer-to-function or as an implicitly declared closure depends on number and type of declarations of function with name *id-expression* have been seen prior to its parsing.

This makes the meaning of the program sensitive to the order of declarations, and even a change as slight as the order of included headers may alter the behavior of the program.

2.1. Example program

Let's consider the program comosed of the following files:

File: container1.hpp

```
namespace cont
{
    class C1 //container
    {
        /* ... */
    };

    bool empty(C1 const&);
}
```

File: container2.hpp

```
namespace cont
{
    class C2 //container
    {
        /* ... */
    };

    bool empty(C2 const&);
}
```

File: remove_empty.hpp

```
#include <algorithm>

namespace cont
{
    template<typename I>
    I remove_empty(I first, I last)
    { return std::remove(first, last, empty); } //P: point of checking of visible overloads of empty.
}
```

File: main.cpp

```
#include <vector>
#include "container1.hpp"
#include "container2.hpp"
#include "remove_empty.hpp"

namespace my_cont
{
    class MC
    {
        /* ... */
    };

    bool empty(MC const&);
}

int main()
{
    std::vector<cont::C1> vc1(10);
    cont::remove_empty(vc1.begin(), vc1.end()); // 1

    std::vector<cont::C2> vc2(10);
    cont::remove_empty(vc2.begin(), vc2.end()); // 2

    std::vector<my_cont::MC> vmc(10);
    cont::remove_empty(vmc.begin(), vmc.end()); // 3
}
```

In the case of the above program there will be two overloads of `empty` visible though normal lookup at the point of use of `empty` as template parameter (marked as `//P`). This according to the wording presented in the paper lead to generation of the code equivalent to:

```
template<typename I>
I remove_empty(I first, I last)
{ return std::remove(first, last, [](auto&&... args) { return empty(std::forward<decltype(args)>(args)...); }); }
```

The closure generated in place of `empty` identifier, performs the unqualified call of the function, so it will consider overloads of `empty` found by ADL. As consequence both lines marked as `//1`, `//2` and `//3` will compile, while for the `//3` line the `my_cont::empty` declaration will be found though ADL.

However if the order of the includes in the `main.cpp` will be changed to:

```
#include "container1.hpp"
#include "remove_empty.hpp"
#include "container2.hpp"
```

At the point of `//P` only one declaration one of `empty` will be visible, so according to the wording paper, this will lead to generation of following code.

```
template<typename I>
I remove_empty(I first, I last)
{ return std::remove(first, last, &empty); }
//As the empty is not-overloaded, the &empty is unambiguous
//and produces bool (*)(cont::C1 const&)
```

This resolution is selected to preserve current compatibility that resolves `function-id` pointing to single not overloaded function, to function pointer. That means that the lines `//1` and `//3` will not longer compile as both `cont::C2` and `my_cont::MC` will not be accepted as argument for `bool (*)(cont::C1 const&)`.

For similar reasons only line `//2` will compile, if the header will be placed in following order:

```
#include "container2.hpp"
#include "remove_empty.hpp"
#include "container1.hpp"
```

Finally the program will produce error mentioning not unresolved name `empty`, if the header will be ordered as follows:

```
#include "remove_empty.hpp"
#include "container1.hpp"
#include "container2.hpp"
```

2.2. ODR Violation

The above example showed how the semantics of the same instantiation of the `filter_empty` changes depending of the order of

the headers in the program. In addition they expose a deeper problem with proposed resolution: depending on the order of the header included in the translation units, the semantic of the same instantiation of `filter_empty` will change, which according to the standard (14.6.4.1 [temp.point] p8) leads to the ill-formed programs.

2.3. Summary

As the above example shows up, the use of the feature of passing overloaded functions sets as template arguments inside of function template lead to fragile code that changes its meaning and validity depending on the subtle changes in the order of declarations (headers). This is caused by the fact that generation of closure depends only on normal lookup and ignores ADL and two phase lookup, that were created to avoid such problems in generic code in first place.

3. Scope and usability

The aim [P0119R1](#) is to make following code to be well formed when multiple overloads of function `f` are found:

```
template<typename I>
void apply_f(I first, I last)
{
    transform(first, last, f);
}
```

Although the author believes that we should aim to provide more generic extension that will allow us to transform any function name to an closure.

3.1. Argument Depended Lookup

Let consider following function example:

```
template<typename I>
I remove_empty(I first, I last)
{ return std::remove_if(first, last, [](auto const& arg) { return empty(arg); }); }
```

I think that it would be reasonable to expect that invocation of above function (and function itself) can be replaced by the `std::remove_if(first, last, empty)` without any loss of efficiency and readability.

To make the call `std::remove_if(first, last, empty)` equivalent to `remove_empty(first, last, empty)` we need to make sure that closure will be generated in place of `empty`. According to rules proposed in [P0119R1](#) this will only take place if declaration(s) of `empty` found though normal lookup will represent overload set (multiple overloads or function template). But it is possible that at the point of declaration of `remove_empty` function, even no declaration of `empty` may be visible, as the unqualified call to `empty(arg)` allow overloads to be found by ADL at point of `remove_empty` instantiation.

3.2. Default arguments

In addition to usual means, like declaration of multiple overloads of single function name or declaring it as function template, there is one additional way to create a function that can be called with more than one set of arguments. We can declare single function with default parameter:

```
void increment(int& i, int n = 1)
{ i += 1; }
```

Let's imagine that our goal is to produce a `increment_all` function that will increment by all elements in given range of integers. We could implement it using the `std::for_each` with `increment` passed as argument. Implementation using proposed feature may look like:

```
template<typename I>
void increment_all(I first, I last)
{
    std::for_each(first, last, increment);
}
```

At the first glance, we could expect that in place of functor argument the following closure will be generated `[](auto&&... args) { return increment(std::forward<decltype(args)>(args)...); }` and it will be invoked with only one `int& i` argument. However this will not be the case, because only one declaration of `increment` function is visible at the point of `increment_all` and this identifier will be resolved to `void (*)(int&, int)` function pointer.

3.3. Avoiding indirection through function pointers

Even in situation when the `std::find_if(first, last, func)` compiles correctly today, function pointer is be passed to the

invoked algorithm and indirect call is performed for each element of the collection. As this may negatively affect performance, the user may want to pass functor object instead.

Despite its usefulness, such transformation change cannot be performed unconditionally, as existing code may depend on additional semantic that is provided by the function pointers when compared to closure object: nullability and assignability from pointer to function with same signature.

4. Acknowledgements

I would like to thank Philipp Juschka for originally proposing the idea of lifting expression and Andrew Sutton for continuing work on this features.

Andrzej Krzemiński offered many useful suggestions and corrections to the proposal.

Special thanks and recognition goes to Sabre (<http://www.sabre.com>) for supporting the production of this proposal, and for sponsoring author's trip to the Oulu for WG21 meeting.

5. References

1. Andrew Sutton, "Overload sets as function arguments" (P0119R1, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0119r1.pdf>)
2. Philipp Juschka, "Lifting overload sets into function objects" (N3617, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3617.htm>)